

JavaScript's Promise.all()

silvermedal.cc/en/challenges/more-javascript/javascripts-promiseall

Silver Medal Academy

All LessonsLessonCode EditorMy Profile

from Arrays

Use Destructuring Assignment with the Rest Operator to Reassign Array Elements

Use Destructuring Assignment to Pass an Object as a Function's Parameters

Create Strings using Template Literals

Write Concise Object Literal Declarations Using Simple Fields

Write Concise Declarative Functions with ES6

Use class Syntax to Define a Constructor Function

Use getters and setters to Control Access to an Object

Understand the Differences Between import and require

Use export to Reuse a Code Block

Use * to Import Everything from a File

Create an Export Fallback with export default

Import a Default Export

Create a JavaScript Promise

JavaScript's Promise.all()

Regular Expressions

Intermediate Algorithm Scripting

Intermediate Algorithm Scripting Projects

Introduction to the Web APIs and the DOM

The Document Object Model (DOM)

Advanced JavaScript Projects

Claim Your JavaScript Algorithms and Data Structures Certificate

Beginning Web Development (phase II term 5)

Advanced Web Development (phase II term 6)

Front End Libraries (phase III term 7)

JavaScript's Promise.all()

Given an array of promises, Promise.all returns a promise that waits for all of the promises in the array to finish. It then succeeds, yielding an array of result values. If a promise in the array fails, the promise returned by all fails too, with the failure reason from the failing promise.

Read more about it, and see examples in this [MDN article](#).

Implement something like this yourself as a regular function called Promise.all. Remember that after a promise has succeeded or failed, it can't succeed or fail again, and further calls to the functions that resolve it are ignored. This can simplify the way you handle failure of your promise.

The idea here is that your function will accept an array of Promises (arrayOfPromises), and return a single Promise which resolves to an array of values. Essentially your response promise should iterate through myPromises, adding the resolved value of each one to the responseArray.

To do this, you should execute the then function for each promise in turn, adding its value to the responseArray in its callback function. Don't resolve the response of your new promise until the final iteration through all the sub-promises.

You'll probably want to use a counter, starting with the length of the promises array, to keep track of how many promises have been processed so far, and resolve only when the counter is down to zero. If any of the inner promises reject, you should catch this and run reject.

Run tests (ctrl + enter)

Reset your code

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

function Promise_all(arrayOfPromises) {
 return new Promise((resolve, reject) => {

 let responseArray = [];
 if (arrayOfPromises.length == 0)
 resolve(responseArray);
 let remaining = arrayOfPromises.length;

 // Enter your code below here:

 for (let i = 0; i < arrayOfPromises.length; i++)
 {
 arrayOfPromises[i].then(value => {
 responseArray[i] = value;
 remaining--;
 if (remaining == 0)
 resolve(responseArray);
 }).catch(reason => reject(reason));
 }

 // resolve(responseArray);
 // Only change code above this line.
 });
}

// The following code will be used to test your function.

soon = function(val) {
 return new Promise(resolve => {
 setTimeout(() => resolve(val), Math.random() *
500);
 });
}

let promise1 = soon(1);
let promise2 = soon(2);
let promise3 = soon(3);
let promiseArr = [];
promiseArr.push(promise1);
promiseArr.push(promise2);
promiseArr.push(promise3);

let promiseRejected = Promise.reject("X");
let promiseArrBad = [...promiseArr];
promiseArrBad.push(promiseRejected);

I found a previous solved solution. Loading now.

CourseApp
Home
Course
Profile
Admin
Logout

Admin: Quiz #1

Testing quiz

30

1

80

Save

Quiz description
Max time (m)
Max tries (#)
Min score to pass (%)

+ Add

Save

Cancel

Questions:

Order

Type:

Multiple choice, single answer

Question:

MathJax Instructions

Enter question...

Choices:

☐ Enter text...
☐ Enter text...
☐ Enter text...
☐ Enter text...

New image

Browse

New video

Browse

</>
[Enter question details.](#)

le Text Inputs as Form Controls

lane: Font Awesome icon by adding `<i class="fa fa-paper-plane">` ton element.

ld a class of `.form-control`. Give your form's submit button the so give this button the Font Awesome icon of `fa-paper-plane`.

rea>, and `<select>` elements with the class `.form-control` have a

Run tests (ctrl + enter)

Reset your code

```

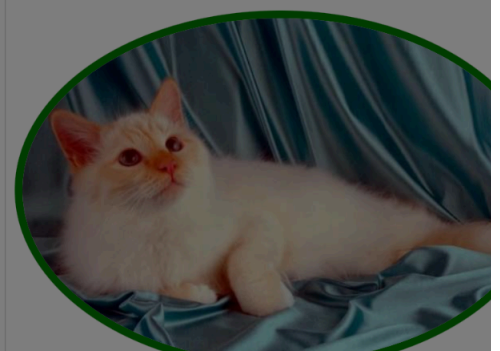
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
<link
href="http://redrockcodecamp.org/educationMaterials/frameworks/fonts/lobster/css/lob
ster.css" rel="stylesheet" type="text/css">
<style>
h2 {
font-family: Lobster, Monospace;
}
.thick-green-border {
border-color: green;

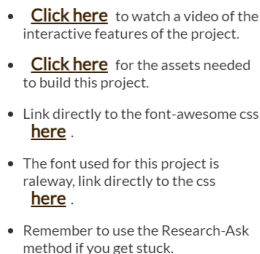
```

Happy Coding!



CatPhotoApp







Home

Welcome to the home page

Hello Dave Norton!

Featured Course



Learn to Code

Learn to code in just a few weeks

[Click to learn more](#)

My Courses

Ad Goes here.

Ad image

Buy something!

Random advertisement

[Click to learn more](#)

Subscribe today!

Enter e-mail

[SUBSCRIBE NOW](#)



Welcome!

 Login with Google+
(test users only)

Or sign in manually:

Username

Password

Login

Register



Subscribe today!

SUBSCRIBE NOW

[Home](#)

[About Us](#)

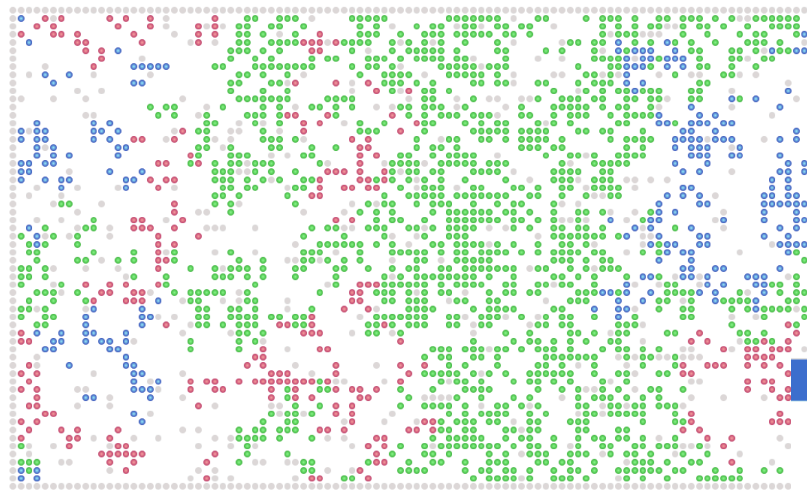
[Helpful Info](#)

[What We Offer](#)

[Contact](#)

CALL US: (801) 857-5309

Copyright © 2022 | Created by Norton Solutions



```
1 import { gridToString, prepareGrid, World } from "../app/world.js";
2 import { WorldAnimator } from "../app/worldAnimator.js";
3
4 var app = {
5   load() {
6     const ROWS = 60;
7     const COLS = 100;
8
9     var itemOptions = {
10      empty: {
11        symbol: "e",
12        buildChance: 30
13      },
14      wall: {
15        symbol: "w",
16        buildChance: 10
17      },
18      plant: {
19        symbol: "p",
20        buildChance: 20,
21        ableToMoveOn: ["empty"],
22        chanceToSurvive: 74,
23        chanceToReproduce: 77,
24        chanceToMove: 52
25      },
26      animal: {
27        symbol: "a",
28        buildChance: 20
29      }
30    };
31  }
32};
```

offer



Petaluma

Vacation Home

\$ 3452135

3215 Sqft 3 Bedrooms 4 Bathrooms

offer

SEE MORE

Are you buying or selling?

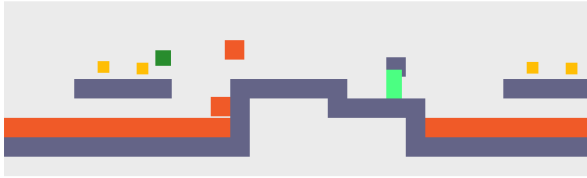
Enter e-mail

SUBSCRIBE NOW

Copyright © 2019 | This site is made with ♥ by Dave

CALL US: 652 345 3222 11

redrockcodecamp.org/prop...



Elements Console Sources Network Performance Memory Application Privacy and security Lighthouse Recorder

Page Workspace >> platform-game.html X

top

redrockcodecamp.org

educationMaterials/work...

platform-game.html

```
197 Monster.prototype.size = new Vec(0.8, 0.8);
198
199 class Lava {
200
201   constructor(pos, speed, reset) {
202     this.pos = pos;
203     this.speed = speed;
204     this.reset = reset;
205   }
206
207   get type() { return "lava"; }
208
209   static create(
210     if (ch ==
211       return
212     } else if
213       return
214     } else if
215       return
216   )
217 }
218
219 Lava.prototype.siz
220
221 class Platform {
222
223
```

20 ms interval 1257 generations

worldItemFactory.js worldAnimator.js X

```
1 import { WorldView } from "../worldView.js";
2
3 class WorldAnimator {
4
5   constructor(world) {
6     this.interval = 20;
7     this.world = world;
8     this.worldView = new WorldView(this.world.toStringGrid(), this.interval);
9   }
10
11   start() {
12     this.generation = 0;
13
14     window.setInterval(() => {
15
16       if (! this.worldView.paused) {
17         this.generation += 1;
18         let beforeTurn = new Date();
19         this.world.turn();
20         this.turnTime = new Date() - beforeTurn;
21         this.worldView.render(this.world.toStringGrid(),
22           {
23             interval: this.interval,
24             generation: this.generation,
25             turnTime: this.turnTime
26           });
27       }
28     }, this.interval);
29   }
30 }
```



Silver Medal CodeCamp

Issued April 1, 2025

This certifies that

David Norton

has successfully completed RedRock CodeCamp's

JavaScript Algorithms and Data Structures Certificate

Representing approximately 300 hours of coursework in JavaScript
including: Data Structures, Debugging, Algorithm Scripting, DOM, and
Object-Oriented Programming
